



UNIVERSITY OF WEST ATTICA
SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING AND
COMPUTERS

LABORATORY

SIGNAL & SYSTEMS

LABORATORY EXERCISE #1 – LEARNING
OF MATLAB

SCHEDULE YEAR 2020-2021

LABORATORY EXERCISE 1

LEARNING MATLAB

The Signals and Systems laboratory is conducted in the program environment MATLAB using both the basic core of the package that provides a multitude of ready-made built-in functions and rich graphics as well as some of its toolboxes. In a short time, the user can use the ready-made subroutines to solve problems from various applications.

The main reasons that have made MATLAB one of the most popular scientific software packages are the following:

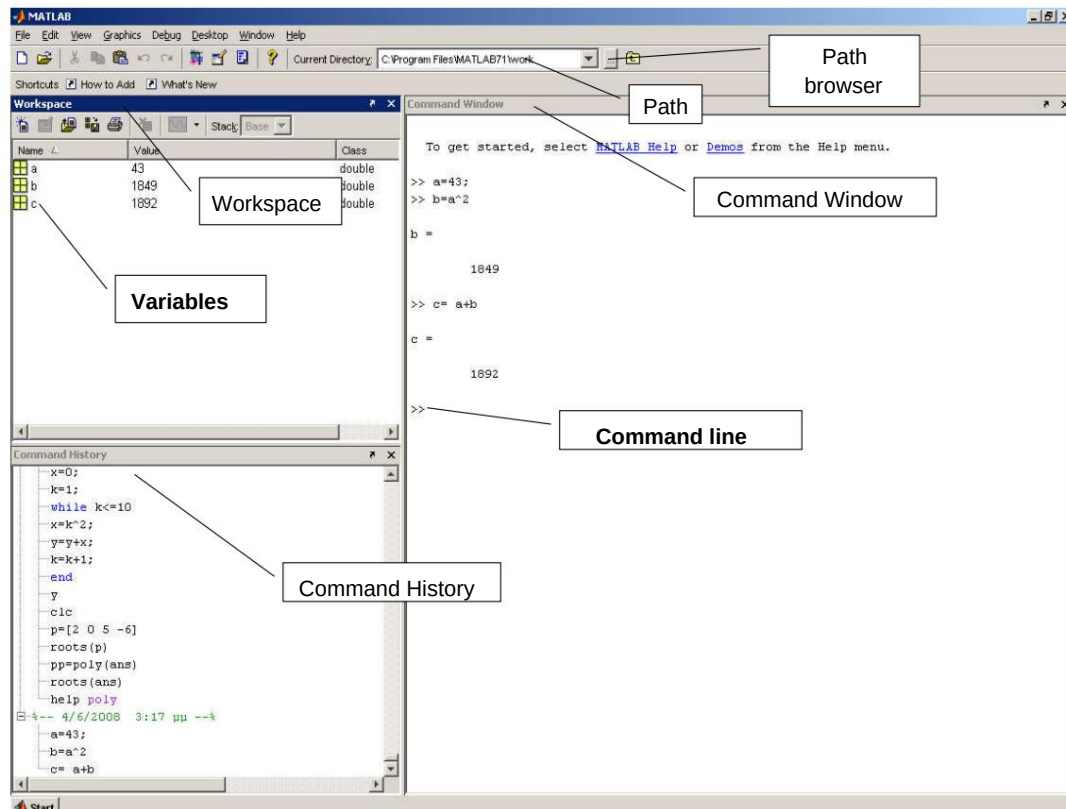
- a) Its environment is user-friendly
- b) Provides direct graphical visualization capabilities
- c) It has a wealth of built-in functions
- d) Provides the ability to add user-written functions
- e) Programming in MATLAB is simple
- f) Includes a variety of tools from various scientific areas

Work environment

The Matlab working environment includes some windows, which are:

- The Command window is the window that contains the command line. There we type the commands we want to execute
- The Workspace is the workspace in which all variables and tables created through the commands we give on the command line are stored.
- Command History, which shows the commands executed since the program was started, as well as commands given while using the program on previous occasions.

- Also, at the top, the Path line is visible, which shows the path of the directory that Matlab sees, as well as a button that leads to the Path Browser.
- In addition to these windows, there is also the Matlab editor, where the writing the program code.



Matlab offers two alternative ways of working. Either through the Command Window where the user gives individual commands, can do some tests and get some results immediately, or through scripts, that is, programs, which the user writes in the editor environment.

Scripts consist of a series of commands, just like programs written in various other programming languages, and which are executed either directly from the editor by giving the Run command or from the Command Window by writing the name of the script. Scripts are code files that are saved with the .m extension and are usually saved in the work file within the Matlab directories as this location is selected by default.

However, the user can work from any location as long as they specify the path in which Matlab will search for the files it needs when executing a program.

The path can be defined either by writing the Path in the line shown at the top of the screen, or through the Path Browser.

1. BASIC MATLAB FUNCTIONS

To enter MATLAB, double-click on the MATLAB icon and to exit, type quit. The MATLAB prompt is `>>`.

Simple arithmetic operations

MATLAB uses the operators `+`, `-`, and operations. `*` and `/` for the four numeric For example:

```
>>3/5
```

```
>>3 + 5 + 2
```

```
>>3^3
```

MATLAB, in addition to the left division operator `/`, also has a right division operator:

```
>>2/4
```

```
>>2\4
```

For more complex expressions, parentheses are used in the usual way:

```
>>2^5 + 4*(33 - 2*(6+2/7))
```

Built-in functions

MATLAB provides us with a number of built-in functions such as square root, exponential and logarithmic functions, trigonometric and inverse trigonometric functions, etc.:

```
>> sqrt(2)           % square root
>> exp(1)            % exponential function
>> log(exp(1)) % natural logarithm
>> log10(10^2) % decimal logarithm
```

% Trigonometric functions

```
>> pi                % the constant p
>> sin(pi/4)         % semitone
>> cos(pi/2)         % cosine
>> tan(pi/4)         % tangent
>> asin(0.5)         % arc sine
>> atan(1)           % arc tangent
```

The '%' symbol is used to enter comments. If the angle is given in degrees, then we convert it to radians by multiplying by $\pi/180$.

Although sometimes the result presents an error due to the numerical approximations of digital computers, as in the above example of calculating $\cos(\pi/2)$ which should have given zero, we should not generalize and perceive all small numbers as zero!

Constants and variables

MATLAB allows us to give constants and variables names of our choice. For example, suppose we want to calculate the following:

$$\sin(60\pi/180)^2 + \cos(60\pi/180)^2$$

Using constants and variables, the calculation can be done as follows:

```
>> theta = 60*pi/180;
>> a = sin(theta);
>> b = cos(theta);
```

```
>> a^2 + b^2
```

The symbols theta, a and b represent constants or variables depending on whether they are allowed to change later or not. The symbol ans is a variable and can also be used in further calculations as in the example:

```
>> 60*pi/180
```

```
>> sin(ans)
```

Format

MATLAB provides the ability to display numbers with different numbers of digits depending on the desired precision. Of course, the internal representation of numbers is independent of the display format. The default format in MATLAB regarding significant decimal digits is given by the 'format' or 'format short' command which displays up to four decimal digits while the default for line spacing is given by the 'format loose' command. For greater precision we can use the 'format long' and 'format long e' commands while to eliminate blank lines, which is recommended, we use the 'format compact' command.

Examples:

```
>> compact format
```

```
>> pi
```

```
>> long format
```

```
>> pi
```

```
>> format long e
```

```
>> pi
```

```
>> x = 2? y = 3; z = x^2 + y^2 + x*y + x + y
```

```
>> short format
```

In the last example we see that the Greek semicolon at the end of a line prevents the result (the values of the variables x and y) from being displayed. The value of the variable z is displayed because there is no ';' symbol in

end of line. More information about the format can be found by typing 'help format'.

SERIES - VECTORS

In MATLAB, an array is defined as an ordered collection of numbers enclosed in square brackets [...] with elements separated by either spaces or commas.

```
>> odd = [1 3 5 7 9 11 13 15 17 19]
>> even = [2,4,6,8,10,12,14,16,18,20]
>> decade = [1.0 1.2 1.4 1.6 1.8 2.0]
```

The elements of the sequence are identified by positional indices (natural numbers) starting from 1:

```
>> odd(5)
>> even(1)
```

The number of elements in a series is calculated by the length function of MATLAB:

```
>> length(even)
```

The clear command resets (erases from memory) the sequence:

```
>> clear even
>> even
??? Undefined function or variable 'even'
```

Operations with series

Let the series $A = [a_1 \ a_2 \ \dots \ a_n]$ and $B = [b_1 \ b_2 \ \dots \ b_n]$. The addition and subtraction of the two series are defined as follows:

$$A + B = [a_1 + b_1, a_2 + b_2, \dots, a_n + b_n]$$

$$A - B = [a_1 - b_1, a_2 - b_2, \dots, a_n - b_n]$$

For example:

```
>> odd + even
```

```
>> even - odd
```

In the case where the elements of the series are at equal distances, then the detailed introduction of the series is not needed but only the first element, the step and the last element with the separator symbol ':'. For example, the above odd and even series can also be defined as follows:

```
>> odds = 1:2:19
```

```
>> even = 2:2:20
```

When the step is 1 then it can be omitted while negative and fractional steps are also allowed:

```
>> natural = 1:10
```

```
>> inv_odd = 19:-2:1
```

```
>>decade = 0:0.1:1
```

Definition of series multiplication operation:

$$A .* B = [a_1b_1, a_2b_2, \dots, a_nb_n]$$

The symbol '.*' means element-by-element multiplication. For example:


```
>> odd.*even
```

Definition of division (from left and from right) of rows:

$$A ./ B = [a1/b1, a2/b2, \dots, an/bn]$$

$$A ./ B = [a1 \setminus b1, a2 \setminus b2, \dots, an \setminus bn] \text{ } \dot{=} B ./ A$$

Examples:

```
>> odd/even
```

```
>> odd.\even
```

Definition of raising to a power:

$$A.^m = [a1^m, a2^m, \dots, an^m]$$

Example:

```
>> natural.^2
```

NOTE: Many of MATLAB's built-in functions can be applied to arrays by simply using the array name in place of the argument:

```
>> angle = 0:10:90;
```

```
>> angle = pi*angle/180;
```

```
>> sin(angle)
```

In the above example, simple multiplication or division of a series by a number leads to the corresponding operation of the number with each element of the series.

Row – column vectors

The series of the previous section can also be considered as line vectors (horizontal) with elements the corresponding elements of the series. Although the statement

line vectors can be the same as the row declaration, it is good to enclose the elements of the vector in brackets [] like so:

```
>> odd = [1:2:19]
>> even = [2,4,6,8,10,12,14,16,18,20]
>> N = [1:5]
```

The declaration of a column vector is identical in form to that of a row vector except for the separator which is now either ';' or a newline:

```
>> A=[1;2;3;4;5]
>> B = [2
        3
        5
        7
        11]
```

Converting a column vector to a row vector and vice versa can be done using the inverse vector, which is denoted by an apostrophe:

```
>> At = A'
>> Att = At'
```

Inner vector product

Let there be a row vector $A = [a_1 \ a_2 \ \dots \ a_n]$ and a column vector $B = [b_1; b_2; \dots; b_n]$ with the same number of elements n . The dot product $A * B$ of the two vectors is a pure number and is given by the following equation:

$$A * B = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

For the vectors of the previous section we have (after inverting the column vector A):

```
>> A'*B
```

while the sum of the squares of the first 5 natural numbers will be:

```
>> N*N'
```

TABLES

Arrays in MATLAB are enclosed in square brackets [] and are entered in a simple way. Using the space or comma separators for row elements and the ';' for line breaks, we can define an array as follows:

```
>> A = [1 2 3; 4 5 6; 7 8 9]
```

We can also write each line separately like this:

```
>> A = [1 2 3  
4 5 6  
7 8 9]
```

Access to the elements of the array is done using two pointers in parentheses, the first specifying the row and the second the column:

```
>> A(1,3)  
>> A(3,2)
```

The dimensions of an array are given by the size function:

```
>> size(A)
```

Two arrays A and B with the same number of rows can be listed one after the other. next to each other and create a new table with the same number of rows and columns as their sum in the original tables. This operation is called table tiling and is denoted by [A B]:

```
>> A = [1 2 3; 4 5 6; 7 8 9];
>> B = [1 1 1 1; 2 2 2 2? 3 3 3 3]?
>> C = [AB]
>> size(C)
```

Creating a new table from two tables A and B that have the same number of columns is also possible using the [A; B] function as in the example:

```
>> A = [1 2 3; 4 5 6; 7 8 9];
>> B = [10 11 12; 13 14 15];
>> C = [A; B]
```

To extract a subarray from row x1 to row x2 and from column y1 to column y2 from a table A, we use the form

A(x1:x2; y1:y2). For example, if we want to extract the subarray consisting of the first two rows and columns of C we will have:

```
>> C(1:2,1:2)
```

If we want to extract all rows or all columns, then we don't need to declare it explicitly but just use the ':' symbol.

```
>> C(:,1:2)
>> C(1,:)
>> C(:,1)
>> C(1:3,:)
```

Elementary operations with matrices

The sum of two arrays A and B with the same dimensions $m \times n$ and with elements a_{ij} and b_{ij} respectively, is a new matrix S with dimensions $m \times n$ and elements s_{ij} given by the equation $s_{ij} = a_{ij} + b_{ij}$. Correspondingly, the difference of the matrices leads to a new matrix with elements $s_{ij} = a_{ij} - b_{ij}$.

```
>> S1 = C(1:3,:)
```

```
>> S2 = C(3:5,:)
```

```
>> S = S1 + S2
```

```
>> D = S1 - S2
```

To multiply two matrices A and B, the number of columns of the first must be the same as the number of rows of the second. For example, let A and B have dimensions $m \times p$ and $p \times n$ respectively. Then, the dimensions of the new matrix P corresponding to the product of the two matrices will be $m \times n$ and the

elements of will be given by the equation $p_{ij} = \sum_{k=1}^p a_{ik} b_{kj}$. from me to you

Continuing the previous example, the product of S1 and S2 will be:

```
>> P = S1 * S2
```

A square matrix (such as S1 and S2) can be multiplied by itself:

```
>> S1 * S1
```

Equivalently, for square matrices, we can use the symbol raising to a power ($A^2 = A * A$, $A^3 = A * A * A$, etc.):

```
>> S1^2
```

As in the case of one-dimensional arrays, in the case of two-dimensional arrays (i.e., arrays), a built-in function acts on each element of the array separately:

```
>> angle = [0:10:20;30:10:50;60:10:80]
>> angle = pi*angle/180
>> sin(angle)
```

Also, a short way to define an array of all zeros or ones is by using the functions `ones(m,n)` and `zeros(m,n)`:

```
>> ones(2,3)
>> zeros(2,2)
```

Finally, if we want to create an array of units that has the same dimensions as an array `A`, then we can use the dimensions returned by the `size` function:

```
>> A = [1 2 3 4; 5 6 7 8? 9 10 11 12]
>> [m, n] = size(A)
>> ones(m, n)
>> ones(size(A))
```

Print table

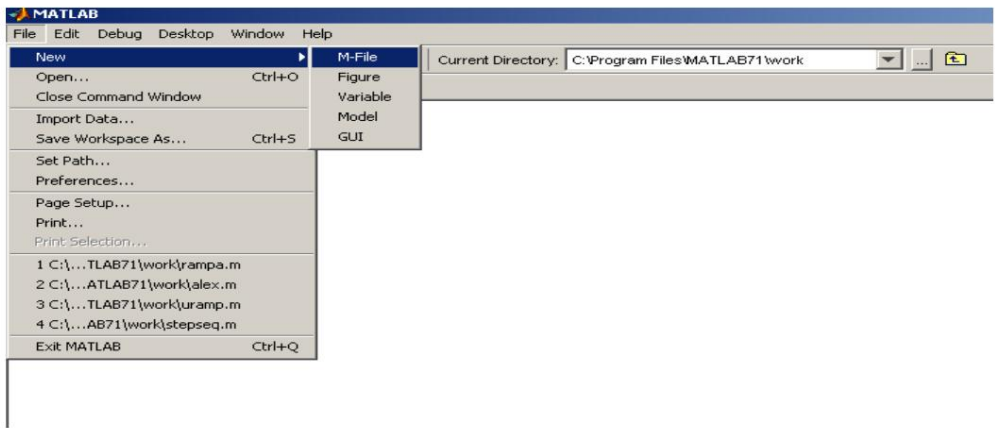
Suppose we want to create a table with the sines of angles 0, 10, 90 degrees. This can be done using the `inversion` and `20, ..., tiling` functions:

```
>> angle = 0:10:180;
>> sine = sin(pi*angle/180);
>> [angle' sine']
```

Creating M files

Matlab has a text editor in which we can write the commands we want, save it and then execute it. The

These files are called M-Files and have the extension *.m. . The image below shows how to open an M-File (from the File->New->M-File menu) as well as an empty M-File on the right side of the image. M-Files must be saved in the active folder. The active folder is usually the work folder. In case we want to save the file in another folder, we must add this folder as active (from the File->Set-Path menu)



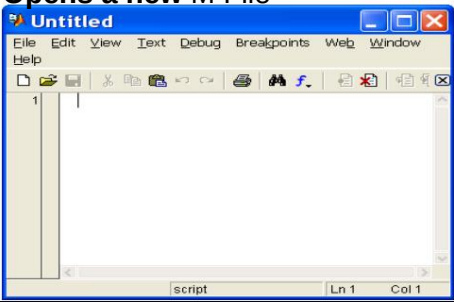
M-Files are divided into two categories: Command files (or scripts) and function files (or functions).

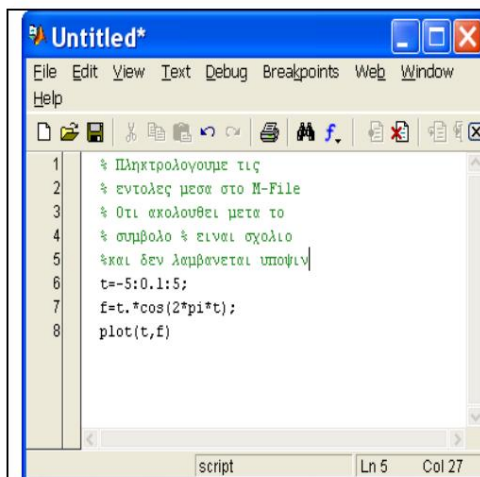
Scripts are text files containing MATLAB commands. Their names must have a .m extension (e.g. name.m) and must not be the name of a command or other file. They are executed either through the MATLAB editor window (Debug -> Run menu) or from the command line.

by simply typing their name (e.g. name) without the .m suffix and pressing Enter. Calling the script executes all the commands it contains. Scripts are suitable for solving problems that require a large number of commands.

Example: Write a program that will plot the function $f(t)$ ()

$t \in [0, 5]$, $f(t) = 5 \sin(t)$

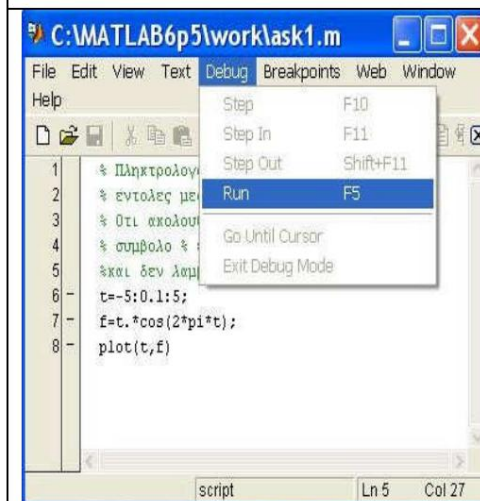
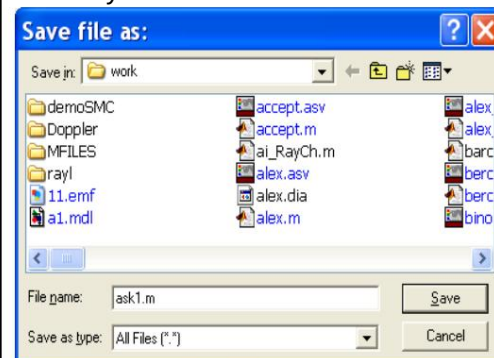
MANDATE	RESULT- COMMENTS
File->New->M-File	Opens a new M-File 



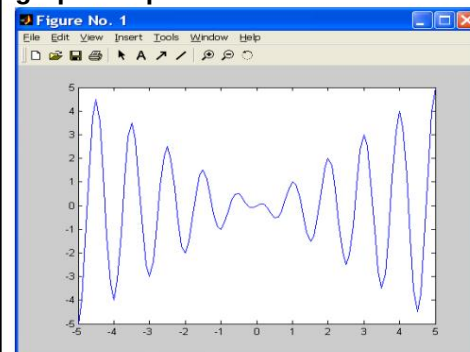
We write the commands we want inside the M-File.
Note that nothing is executed

File->Save

We save the M-File in the active directory named ask1.m

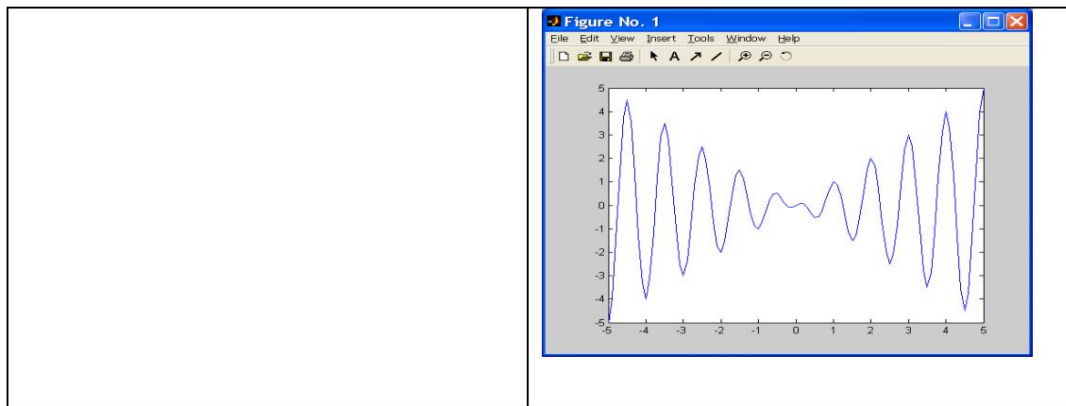


We run the program from Debug->Run and we get as a result the graphic representation we wanted.



>>ask1

Second way of execution. In the Matlab command line we write the name of the script (without the ending m) and press Enter.



The script ask1.m has been saved and we can run it at any time. The positive thing about scripts is that they are very simple. In essence, they are like writing on the command line with the difference that the program is saved and the commands are executed all at once. The disadvantage is that they do not have much flexibility. That is, this particular script will always plot the function $f(t)$ for a time from -5 to 5. This flexibility is added by function files or functions.

Functions are also M-Files, meaning they have the extension .m, and must be saved in the active folder. The difference between functions and scripts is that functions take inputs (arguments) and return outputs. A function always starts (on the first line of the editor) with the following expression:

`function yyy = name xxx 1 2, ..., n`

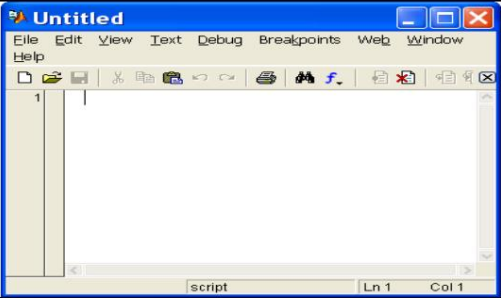
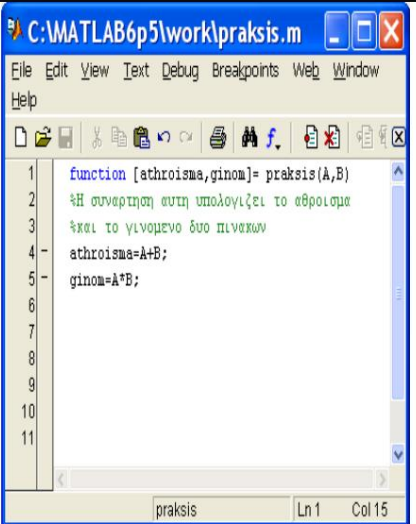
The `yyy` are the outputs returned by the function and `xxx` are the `1 2, ..., n` inputs (arguments) it accepts. In case we have an output the brackets can be omitted. The "name" is the name of the function. Then we can write the main part of the function. In order to run it we must first save it.

Attention: The name you save the function with must be the same as the one you used as the name in the first line.

Finally, unlike scripts, a function is only run from the command line, by writing its name and giving the appropriate arguments (in number and type).

Example: Write a function that takes two arrays as arguments and returns their sum and product.

COMMAND	RESULT - COMMENTS
From the menu we press: File->New->M-File	Opens a new M-File

	
	We start by defining the M-File as function named praksis which accepts the variables A,B as inputs and returns the variables athroysma and ginom as outputs . In then we write the commands that we want in the M-File.
	We save the M-File with the name praksis.m
>> practice	??? Input argument 'A' is undefined. Error in ==> C:\MATLAB6p5\work\praksis.m On line 4 ==> athroisma=A+B; We are running the function from the line commands giving only her name function and as it is normal we receive a message mistake since we did not define the entrances.

<pre>>> C=[1 2; 1 2]; >> D=[2 3; 2 3]? >> praxis(C,D)</pre>	<pre>answer = 3 5 3 5</pre> We run the function from command line giving the name of function and the arguments C and D. The function executes but we on the command line we only get the first result (i.e.) the addition and this saved in ans . we This happens because it is not have defined outputs.	
<pre>>> C=[1 2; 1 2]; >> D=[2 3; 2 3]; >> [PROS, POL]= praxis(C,D)</pre>	<pre>PROS = POL = 3 5 6 9 3 5 6 9</pre>	
	We run the function from command line giving the name of function and determining the arguments C and D and the variables output PROS and POL. This way we get the desired result. The variable PROS contains the result of the addition and the variable POL this of multiplication.	
<pre>>> E=[1 1; 2 3]; >> F=[1 2; 3 4]; >> [PROS, POL]= praxis(ÿ,F)</pre>	<pre>PROS = 2 3 5 7 POL = 4 6 11 16 >></pre>	Here we call the function with 7 different arguments.
<pre>[PROS, POL]= praxis(3,5) PROS =</pre>	<pre>8 POL = 15</pre>	Here we call the function praxis with arguments numbers. We observe the flexibility it offers a carefully written function.

When we click help and the name of a function, the comments we have written in the M-File of that specific function appear on the command line.

COMPLEX NUMBERS

In the following examples we see the ease with which MATLAB handles complex numbers. Notice that in addition to simple operations, we also use some functions such as sqrt (square root) and sin (sine) in the expressions. The abs function calculates the modulus of a complex number or simply the absolute value of a real. The angle calculates the argument (angle) of the complex number in radians, while finally real and imag return the real and imaginary parts of a complex number, respectively.

Examples:

» `z=1-2i` % i is the imaginary unit

`z =`

`1.0000- 2.0000i`

» `w=1-2j` % j is also the imaginary unit

`w =`

`1.0000- 2.0000i`

» `z1=3*(2-sqrt(-1)*3)`

» `z2=sqrt(-2)`

» `z3=6+sin(.5)*i`

» `z4=(z3+w)/z1`

» `i^2`

» `magnitude_z4=abs(z4)` »

`angle_z4=angle(z4)` »

`deg_z4=angle_z4*180/pi` % convert in degrees

» `real_z4=real(z4)` »

`imag_z4=imag(z4)`

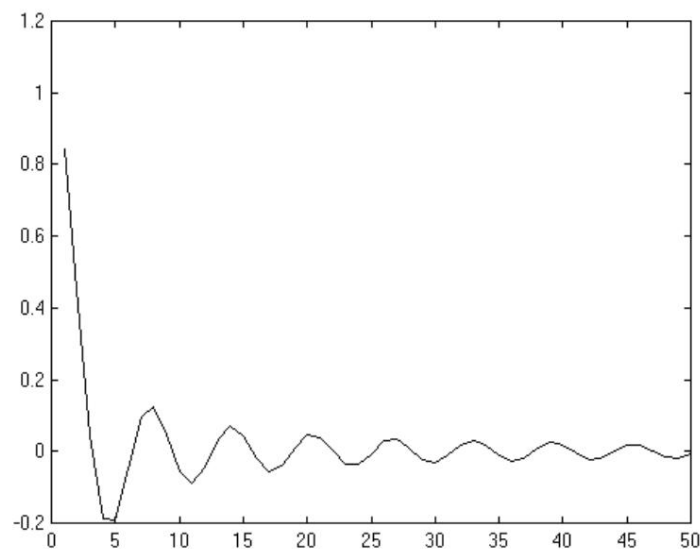
GRAPHICS

(a) In two-dimensional space

The most common function that produces two-dimensional graphs and which we have already used is plot. We will learn about its various forms through examples.

In the simplest case, `plot(x)` plots the elements of vector `x` in terms of their ordinal number. _____

```
»n=1:50;
»an=sin(n)./n;
»plot(an)
```



We notice that `plot` opens a graphics window (*Figure window*), draws axes and places the points by connecting them with straight lines.

If `x` is a matrix, then in the same figure a graph is made for each column _____ of the table :

```
»for n=1:50
a(n,1)=abs(sin(n))/n;
a(n,2)=1/n;
end
»plot(a)
```

In the case where `x` is a vector of complex numbers, then the points representing the complex numbers are plotted with axes corresponding to the real and imaginary parts.

```
>>x=0:0.1:10
>>z=x.^2+x*i
>>plot(z)
```

We also notice that if the Figure window is already open, plot clears it and draws a new graph.

If we use the form `plot(x,y)`, we will get a graphical representation of the elements of `y` against their counterparts of `x`:

```
»x=linspace(-2*pi,2*pi,1000);  
»y=sin(x);  
»plot(x,y)
```

It is possible to draw multiple graphs with different colors together, if we define multiple pairs of vectors in the plot.

```
»x=linspace(0,2,1000);  
»y1=exp(x);  
»y2=2.5.^x;  
»y3=2.^x;  
»plot(x,y1,x,y2,x,y3)
```

Drawing multiple graphs at the same time can also be done using a table. Each column of the table is drawn relative to the reference vector. For example

```
»Y=[y1;y2;y3];  
»plot(x,Y)
```

In the previous examples, we let MATLAB draw the graphs with predefined colors and a specific line type (solid line). It is possible to define our own colors, symbols for the points, and the type of line that connects them. This is achieved by introducing another argument to the plot, after each pair of vectors to be plotted. The new argument is a series of special characters enclosed in apostrophes. The available special characters are shown in the following

array . We can use one or more characters in any order.

Symbol	Color	Symbol	Type points	Symbol	Type line
b	blue	.	.	-	continuous
g	green	the	the	:	dotted
r	red	X	X	-.	combination -.
c	blue	+	+	--	interrupted
m	purple	*	*		
y	yellow	s	square		
k	black	d	rhombus		
w	white	<, >	triangle		
		p	pentagon		
		h	hexagon		

The following commands will create two curves in a graph. The first will be blue, the points will be pentagons and the line will be dotted, while the second will be drawn in red, the points will be circles and the line will be dashed.

```

»x=linspace(-1,1,15);
»y1=asin(x);
»y2=acos(x);
»plot(x,y1,'b:p',x,y2,'r--o')
Y=[y1;y2;y3];
»plot(x,Y)

```

The grid on command adds a grid of lines for better reading of the coordinates, while grid off removes the grid. The grid simply acts as a switch between the two states.

```

»x=linspace(-3,3,150);
»y=exp(-x.^2);
»plot(x,y)
»grid on

```

A graph is framed by a rectangle with the axis divisions. The box off command displays the graph without the rectangle, while box on returns it. The box command works like a switch.

```

»x=linspace(-10,10,150); »y=1./
(1+x.^2);
»plot(x,y)
»box off

```

We can add labels to the horizontal and vertical axes of a graph with the `xlabel` and `ylabel` commands. We can also add a title to the entire graph with the `title` command.

```

»x=-2*pi:.01:2*pi;
»y=atanh(sin(x))-sin(x);
»plot(x,y)
»title('Plot of a function')
»xlabel('Variable x')
»ylabel('Variable y')

```

There is a way to add text to the graph with the `text` command. The general format is `text(x,y,'text')` where (x, y) are the coordinates of the point where the text will be written. With the following commands we will create a graph expression with the curves $\sin(x)$ and $\cos(x)$ and we will add a text at the position $(2.5, 0.7)$.

```

»x=linspace(0.2*pi,150);
»y=sin(x);
»z=cos(x);
»plot(x,y,x,z,':')
»text(2.5,.7,'sin(x)')

```

A simpler way to insert text into a graph is to use the `gtext('text')` command. This takes us to the graph window where a cross appears that can be moved with the mouse. Once we have located the appropriate point, we insert the text by clicking.

```

»gtext('cos(x)')

```

Instead of inserting text into the chart to distinguish the lines, we can insert an overall caption with the `legend` command. Automatically

A rectangle is inserted which contains the labels that we will define for each curve of the graph. We can move this rectangle with the mouse (click and drag) to any point of the graph we wish.

```
»legend('sin(x)','cos(x)')
```

The legend is deleted with the legend off command.

A very important feature of MATLAB graphs is the ability to format the axes. This is achieved with the axis command, which has many special cases (ask for them to appear with the help axis command). The simplest ones are axis off and axis on, which hide and display the axes, respectively.

If we want to have graphs in multiple windows, we can open new graph windows from the File menu by selecting New Figure. Particularly useful is the ability to draw many different graphs with different axes in the same window. The subplot(m,n,p) command divides the active graph window into $m \times n$ positions while simultaneously making the p-position active. The following commands result in the creation of 4 graphs. Note the use of the axis([xmin,xmax,ymin,ymax]) command to define the axes limits. The division of the window is undone with the subplot(1,1,1) command.

```
>>x=linspace(-2*pi,2*pi,1000);
>>y1=sin(x);
>>y2=x;
>>z=y1+y2;
>>w=y1.*y2;
>>subplot(2,2,1)
>>plot(x,y1),axis([-2*pi,2*pi,-1,1]),title('sin(x)')
>>subplot(2,2,2)
>>plot(x,y2),axis([-2*pi,2*pi,-2*pi,2*pi]),title('x')
>>subplot(2,2,3)
>>plot(x,z),axis([-2*pi,2*pi,-2*pi,2*pi]),title('sin(x)+x')
>>subplot(2,2,4)
>>plot(x,w),axis([-2*pi,2*pi,-2*pi,2*pi]),title('x*sin(x)')
```

If we want to examine an area of the graph better, we can expand it with the zoom on command. By giving the command, the focus is activated. In the graphics window, by clicking the left mouse button we expand the area while by clicking the right mouse button we move away. We can also delimit an area that we will expand with click and drag. The zoom off command suspends focus. The following commands draw (a) the graph of the function generated by the following commands and (b) expand a region of the graph.

```
»x=linspace(-2,2,3000); »y=sin(1./
x);
»plot(x,y)
»grid on
```

Some other graphing commands that replace plot in special cases are:

loglog: The axes scales are logarithmic.

semilogx: The x-axis scale is logarithmic.

semilogy: The scale of the y-axis is logarithmic.

area(x,y): The area between the curve and the x-axis is plotted.

Example: With the following commands we sketch an area under the curve $y = \exp(x)$. Notice the hold on command which keeps the previous graph in the window so that the new one can be drawn on top of it. To stop the hold, we give hold off.

```
»x=-1:0.001:1;
»y=exp(x);
»plot(x,y)
»hold on
»x1=0:0.01:0.4;
»y1=exp(x1);
»area(x1,y1)
```

Polar coordinate graphs are created with the command `polar(t,r)` where `t` is a vector with angles in rad and `r` is a vector with distances. The format of the graph (colors, points and lines) can be adjusted

with a set of special characters in exactly the same way we saw in the plot command.

Example: We will graph functions given in polar coordinates.

$r = \theta$ (Archimedes spiral) and $r = \sin(3\theta)$ (trefoil)

The commands we will give to get the two graphs in the same window are:

```
»theta=linspace(0.5*pi,1000);
»r1=theta;
»r2=sin(3*theta);
»subplot(1,2,1)
»polar(theta,r1,'.')
»subplot(1,2,2)
»polar(theta,r2,'.')
```

A simple way to create graphs of functions without having to create a vector of values of the independent variable is given by the command `fplot('function', [xmin,xmax,ymin,ymax])` which plots the function that we will define within the limits for the two axes that we will also define (the limits of the y-axis ymin and ymax are not mandatory).

Example:

```
»fplot('2*csch(sec(x))',[-8,8,-2,2])
```

Note: If we want to open multiple graphics windows through commands belonging to an M-file, we should use the figure command. The `figure(n)` command selects the graphics window with number n as the active graphics window.

(b) Three-dimensional graphics

To create curves in three-dimensional space (line plots), we use the `plot3` command. Its syntax and features are the same as `plot` except that here we define triples of vectors instead of pairs. The general form is:

```
plot3( xyz_string, _ , '1' , xyz_string' ) _
```

where string is the sequence of special characters for setting the color, line and points. This principle extends to all the commands that accompany it. For example, the text command accepts 3 coordinates, while for a label on the z-axis we use the zlabel command. The grid, box, subplot etc. commands have exactly the same function.

Example: We will draw the curves defined by the vector functions

$$\vec{r}_1(t) = (\sin t, \cos t, \sin^2 t) \quad \text{and} \quad \vec{r}_2(t) = (\sin t, \cos t, t^2).$$

```
»t=linspace(0,10*pi,1000);

»subplot(1,2,1);

»plot3(sin(t),cos(t),(sin(t)).^2),title('sin(t),cos(t),(sin(t)
).^2')

» box on

»subplot(1,2,2) ;

»plot3(sin(t),cos(t),t),title('sin(t),cos(t),t')

»box on
```

Another form of graphical representation in three-dimensional space are surfaces that represent functions of two variables $f_{xy}(x, y)$. Their production process is as follows:

1. We define two vectors x and y which have points in the range of values of the corresponding variables for which the graph will be made.
2. With the meshgrid command we convert the two previous vectors into matrices X and Y where the rows of X are repetitions of the vector x and the columns of Y are repetitions of the vector y .
3. We calculate the values of the function f for the corresponding elements of both of tables X and Y .
4. The graphic representation is done using the mesh command.

The following commands are an example of designing a surface in the form of a grid:

```
»x=linspace(-7.5,7.5,50);

»y=x;

»[X,Y]=meshgrid(x,y);
```

```
»R=sqrt(X.^2+Y.^2)+eps;  
»Z=sin(R)./R;  
»mesh(X,Y,Z)
```

The surf command is exactly the same as the mesh command, with the only difference being that the surf fills the surface with colors. Try the command

```
»surf(X,Y,Z)
```

Symbolic expressions

MATLAB has a collection of functions under the name Symbolic Math Toolbox that serve as tools for performing symbolic operations such as solving algebraic and differential equations, deriving and integrating functions, finding determinants and characteristic roots, etc.

In every symbolic expression it is important to first declare the variables that take part in the expressions as symbolic variables. This is done with the sym (for a single variable) and syms (for many variables together) functions. For example, with the following commands, we declare a variable x as symbolic and then use it to calculate the derivative of the function cosx.

```
»x=sym('x')  
»diff(cos(x))
```

With the following commands, we declare 4 symbolic variables together, construct a symbolic array with them, and then calculate its determinant.

```
»syms('a','b','c','d')  
»M=[a,b;c,d]  
»det(M)
```

(a) Numerators and denominators of rational expressions

When we have a complex expression with fractions that we want to convert into To convert an expression to an integer, we use the numden command. This returns the numerator and denominator of the expression separately. For example, if we want to convert the expression

which we denote the factorial (!) since MATLAB cannot recognize it otherwise. To denote infinity, we use the MATLAB variable `inf`.

(c) Derivation

The derivative of functions is done with the diff command. Its different forms are:

- **diff(expression):** Generates the expression with respect to one of the symbolic variables

It is preferable to use it when the expression includes only one _____

variable.

- **diff(expression, n):** Generates the expression n times.
- **diff(expression, variable, n):** Generates the expression n times with respect to the variable that we will define.

For example, we will calculate the first and second order partial derivatives of the function of 2 variables

$$(,) f_{xy} e^{y^2 x}$$

»syms xy

»f=exp(-2*x)*sin(y)

»df_dx=diff(f,x)

»df_dy=diff(f,y)

Note that the diff function is used in the core of MATLAB to calculate the differences of consecutive elements of a vector or columns of a matrix. MATLAB decides which version of the command to use since it is able to distinguish between symbolic and numerical expressions.

(d) Completion

The command with which we can calculate integrals is int. It also has various forms, but the most complete ones are the following:

int(expression, variable): Calculates the indefinite integral with respect to the symbolic variable we will define. There is a case where either the integral is impossible to give in closed form, or the function is so complex that MATLAB cannot calculate the antiderivative.

int(expression, variable, lower bound, upper bound): Calculates the definite integral over some integration interval.

For example, we will calculate the integrals:

$$\int \frac{3x}{(x^2+2)^5} dx \quad \int \frac{x^2}{(2x^2+5)^5} dx \quad \int \frac{1}{4} \tan^{-1} \frac{x}{2} dx$$

```
»syms x
```

```
»g1=(3*x+7)/(x^2+2*x+5)^2
```

```
»int(g1,x)
```

(e) Solving equations

The solve function is used to find the roots of a symbolic expression. We can specify a variable with respect to which the solution will be performed (recommended).

```
»syms abc x
```

```
»solve(a*x^2+b*x+c,x)
```

```
answer =
```

```
[ 1/2/a*(-b+(b^2-4*a*c)^(1/2))] [ 1/2/a*(-b-  
(b^2-4*a*c)^(1/2))] »pretty(ans)
```

$$\begin{bmatrix} \frac{1}{2} \left[-b + (b^2 - 4ac)^{1/2} \right] \\ \frac{1}{2} \left[-b - (b^2 - 4ac)^{1/2} \right] \end{bmatrix} \frac{1}{a}$$

Algebraic commands

A large number of commands support the manipulation of symbolic expressions, applying algebraic operations to analyze, simplify, and optimize their form. The main ones are the following:

- `expand(expression)`: expands into simple polynomials an expression that is either a product of polynomials or a (positive and integer) power of a polynomial.
- `factor(polynomial)` factors simple polynomials
- `mod(m, n)` gives the remainder of dividing m by n.

- **floor(x)** returns the integer that is less than or equal to x. • **ceil(x)** returns the integer that is greater than or equal to x.
- **numeric(s)**: Converts a symbolic expression s (which does not contain symbols) to an arithmetic one. H s can also be a symbolic array. •
- simple(expr)**: Simplifies a symbolic expression by converting it to another simpler one by factoring, expanding, simplifying, using trigonometric and other transformations, etc. The shortest expression in length is returned. • **pretty(f)**: Converts a symbolic expression f into a readable form. • **simplify(f)**: Symbolically simplifies a symbolic expression or a symbolic array. • **expand(f)**: expands into simple polynomials an expression that is either a product polynomials or (positive and integer) power of a polynomial.
- **factor(f)**: Factorization of the symbolic expression f, or an array of integers (in prime factors), or the elements of a symbolic array. • **subc(f, exp, x)**: Substitution in a symbolic expression or array f of a symbolic variable x with the symbolic expression expr. **subc(f,expr)** replaces the default symbolic variable of f with the expression expr. If f contains only one variable and a numeric value v is given in the position expr, then the value f(v) is calculated.

Random numbers

In Matlab we have the ability to generate random numbers that follow specific distributions.

The command that produces a matrix of dimensions NxK with numbers that follow the uniform distribution in the interval [0 1] is **x= rand (N,K)**

For random numbers that follow the uniform distribution in an interval [a,b] we write **x= a+(ba)*rand(N,K) ;**

The command that produces an NxK array with numbers following the normal (normal-gaussian) distribution with mean $\bar{y} = 0$ and variance $\bar{y}^2 = 1$ is **x= randn (N,K)**. For random numbers that follow the normal distribution with mean value $\bar{y}$ and dispersion $\bar{y}^2$ we write **x= $\bar{y} + \bar{y} * \text{randn}(N,K)$;**

To draw the histogram we use **hist(x,k)** where k is the number of subintervals into which we divide the histogram. The **pie(x)** command draws a pie chart from the values of the vector x

Finally, with the command **mean(x)** we find the mean and with the command **var(x)** the variance of a vector x.

Syntactic errors

Some of the most common errors are the following:

• **Parentheses:** if you have forgotten a parentheses, Matlab will report an error of the form you see below, indicating the position (line and column) where the error was detected. Also, the underlined part is a link that takes you directly to the program, at the position where the error was detected.

??? Error: File: C:\MATLAB6p5\work\systrot2.m Line: 20 Column: 26
")" expected, "?" found.

• **Array dimensions:** If the dimensions of the arrays participating in operations do not match, then the following message is displayed. Such messages are also displayed when the operators of the operations are not used correctly. For example, if a is an element and X is an array or a vector, then the following command gives the following result: $C = a / X$; (The correct command would be $C = a ./ X$)
??? Error using ==> /Matrix dimensions must agree.

PRACTICE EXERCISES

ASK1)

- Create a vector b [vector] • Find y [0, 0.1, 0.2, ..., 10] and one
the y (0), (0.2), (0.4), ..., (20)]
the y^4
the inner product of a and b .

ASK2)

- Write a function that will take as an argument a number in radians (rad) and return its value in degrees. • Find how many degrees are in $y/4$ rad

ASK3)

- Write a function that plots $\sin(x)$
sin () function $\frac{y}{x}$
• Draw for the interval $-2y$ to $2y$.

ASK4)

- Write a function that will take a complex number as an argument and will return
 - the phase
 - the magnitude
 - the real part
 - the imaginary part
- Calculate the above quantities for the following complex numbers:
 - $-i$
 - 1
 - e^{3yi}

ASK 5)

- Divide the interval $[0, 2\pi]$ into 500 points
- Draw in this interval (in the same figure) the following signals

$$x \in [0, 2\pi] \quad y = \cos(x)$$

$$y = \sin(x)$$

- Give the title (whatever you want)
- Label the x and y axes (whatever you want)
- Label all the curves with the legend command.
- Draw the two above signals in a second figure

but in 2 different windows.

ASK 6) •

Calculate the partial derivatives of the function $f(x, y, z, t)$

$$f(x, y, z, t) = x^2 y^3 z^4 t^5$$

• Calculate the integral

$$\int_0^t y \, dt$$

• Calculate the double indefinite integral

$$\int \int x e^{xy} dx dy$$

• Calculate the sum

$$\sum_{k=0}^y \frac{x^k}{k!}$$

• Convert the expression into an expression

$$x^3 y^2 = \frac{3}{x^2 y^2} \frac{x}{x^2 y^1}$$

• To be solved

or

equation 3.2

second

degree

:

$$f(x, y, z) = x^2 y^3 z^4$$

ASK 7) Write a function to calculate the roots of a trinomial.